Sistema de Ventas para Cafetería — Documentación técnica

Documentación de referencia del proyecto **Sistema de ventas para una cafetería con catálogo e imágenes 3D**, construido sobre Laravel + MariaDB. Este índice describe el stack, la puesta en marcha y el estado de cada módulo; los detalles de datos, autenticación, catálogo 3D y flujo de compra viven en documentos independientes.

Índice

DOCUMENTO	CONTENIDO
01-base-de-datos.md	Diagrama entidad-relación, diccionario de tablas, claves foráneas, relaciones Eloquent y decisiones de diseño del esquema.
02-roles-y-autenticacion.md	Flujo de autenticación con Breeze, sistema de roles, middleware, mapa de rutas protegidas, credenciales semilla y extensión a nuevos roles (RF-01, RNF-03).
03-catalogo-y-visor-3d.md	Catálogo público, ficha de producto, visor Three.js, atributos `data-*`, carga de modelos GLB, manejo de errores y rendimiento (RF-02, RNF-01, RNF-02).
04-carrito-y-pedidos.md	Carrito en sesión, checkout transaccional, validación de stock y precios, métodos de pago, snapshot de importes, checkout de invitado y confirmación firmada (RF-03, RF-04).
05-administracion.md	Panel de administración, CRUD de productos con subida de imagen y modelos .glb/.gltf, gestión de pedidos y cambio de estado (RF-05, RF-06, RNF-03).
06-inventario-y-reportes.md	Inventario de insumos con entradas/salidas y stock automático, reportes de ventas por período y por producto (RF-07, RF-08).

Stack

COMPONENTE	VERSIÓN INSTALADA	NOTAS
PHP	8.4	El proyecto declara `^8.3` en `composer.json`.
Laravel	13.40.0	Incluye atributos `#[Fillable]` y `#[Hidden]` en modelos.
MariaDB	11.8	Base de datos `cafeteria`, charset `utf8mb4`.
Laravel Breeze	2.4.2	Stack Blade + Alpine.js; autenticación con sesiones.
Tailwind CSS	4.x	Integrado mediante `@tailwindcss/vite`.

Three.js	0.187	`three@^0.187.0`, instalado por NPM y empaquetado con Vite (sin CDN); usado por el visor 3D.
Vite	8.x	Bundler de assets (`npm run build` / `npm run dev`).
PHPUnit	12.5	Suite de pruebas (no se usa Pest).

Dependencias relevantes de `composer.json`: `laravel/framework`, `laravel/tinker`, y en desarrollo `laravel/breeze`, `laravel/boost`, `laravel/pint`, `phpunit/phpunit`. Del lado de Node: `three`, `alpinejs`, `tailwindcss`, `vite`, `laravel-vite-plugin`.

El visor 3D requiere un navegador con WebGL. Si WebGL no está disponible, la ficha muestra un panel de error en lugar del modelo, sin afectar al resto del contenido.

Requisitos previos

- PHP 8.3 o superior con las extensiones habituales de Laravel (`pdo_mysql`, `mbstring`, `openssl`, `ctype`, `json`, `bcmath`, `fileinfo`, `tokenizer`, `xml`).
- Composer 2.
- Node.js 20+ y npm.
- MariaDB 11.8 en ejecución.

Puesta en marcha

1. Dependencias de PHP

```
composer install
```

2. Configuración del entorno

```
cp .env.example .env
php artisan key:generate
```

`.env.example` trae `DB_CONNECTION=sqlite`; edita `.env` para apuntar a MariaDB. La configuración verificada del proyecto es:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=cafeteria
DB_USERNAME=cafeteria
DB_PASSWORD=cafeteria
```

El proyecto usa `SESSION_DRIVER=database`, `CACHE_STORE=database` y `QUEUE_CONNECTION=database`, por lo que las migraciones crean también las tablas `sessions`, `cache` y `jobs`.

3. Crear la base de datos y el usuario

```
CREATE DATABASE cafeteria CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

CREATE USER 'cafeteria'@'127.0.0.1' IDENTIFIED BY 'cafeteria';
GRANT ALL PRIVILEGES ON cafeteria.* TO 'cafeteria'@'127.0.0.1';
FLUSH PRIVILEGES;
```

4. Migraciones y datos semilla

```
php artisan migrate --seed
```

El seed es idempotente (`updateOrCreate`) y crea las cuentas descritas en 02-roles-y-autenticacion.md, además del catálogo de ejemplo (Espresso, Latte y Capuchino) descrito en 03-catalogo-y-visor-3d.md:

ROL	EMAIL	CONTRASEÑA
Administrador	`admin@cafeteria.test`	`password`
Cliente	`cliente@cafeteria.test`	`password`

5. Enlace de almacenamiento público

El visor 3D sirve los modelos desde `storage/app/public`, por lo que el symlink `public/storage` debe existir:

```
php artisan storage:link
```

El enlace está en `.gitignore` y debe recrearse en cada entorno. La carpeta `storage/app/public/productos/` no existe hasta que se sube el primer modelo; consulta la guía de carga de modelos.

6. Assets del frontend

```
npm install
npm run build
```

Para desarrollo con recarga en caliente: `npm run dev` o el script combinado `composer run dev`.

7. Servir la aplicación

```
php artisan serve
```

La aplicación queda disponible en `http://127.0.0.1:8000`. La portada `/` es el catálogo público (`productos.index`). El endpoint de salud responde en `/up`.

> *`composer run setup` automatiza instalación, generación de clave, migraciones y build de assets, pero **no** ejecuta los seeders ni crea el symlink de `storage`. Después de usarlo, ejecuta `php artisan db:seed` y `php artisan storage:link`.*

Verificación rápida

```
php artisan route:list          # mapa completo de rutas
php artisan test                # suite PHPUnit
```

Estado verificado de la suite: **81 pruebas aprobadas y 354 aserciones**, incluidas las de autenticación, registro con rol por defecto, acceso al panel de administración, catálogo/visor 3D, carrito, checkout invitado/autenticado, administración de productos y pedidos, e inventario y reportes de ventas.

Para el visor 3D también conviene comprobar manualmente el render en navegador: las pruebas HTTP validan los atributos `data-*`, no la ejecución de WebGL. La lista de comprobación está en 03-catalogo-y-visor-3d.md. El flujo de compra se verifica de extremo a extremo con los pasos de 04-carrito-y-pedidos.md.

Estado de los módulos

MÓDULO	REQUERIMIENTOS	ESTADO ACTUAL	DOCUMENTACIÓN

1. Autenticación y roles	RF-01, RNF-03	**Completado**: registro, login, logout, recuperación de contraseña, rol `cliente` por defecto, middleware `role` y panel `/admin` restringido.	02-roles-y-autenticacion.md
2. Catálogo y visor 3D	RF-02, RNF-01, RNF-02	**Completado**: portada con catálogo paginado de productos activos, ficha de detalle, visor Three.js interactivo sobre `modelo_3d_path`, estados vacío/carga/error, navbar y layout renovados. La subida de modelos aún es manual (Módulo 4).	03-catalogo-y-visor-3d.md
3. Carrito y pedidos	RF-03, RF-04	**Completado**: carrito en sesión con control de stock, checkout transaccional con revalidación y bloqueo de productos, snapshot de precios, métodos de pago, confirmación firmada y checkout de invitado sin cuenta.	04-carrito-y-pedidos.md
4. Administración	RF-05, RF-06, RNF-03	**Completado**: CRUD de productos (create/read/update/delete) con subida de imagen y modelo 3D, gestión de pedidos con cambio de estado, dashboard con métricas reales y panel protegido por rol.	05-administracion.md
5. Inventario y reportes	RF-07, RF-08	**Completado**: CRUD de insumos, registro de movimientos de entrada/salida/ajuste con actualización automática de stock, reportes de ventas por período y por producto, panel protegido por rol.	06-inventario-y-reportes.md

> *El checkout del Módulo 3 admite invitados: no exige autenticación y guarda `pedidos.user_id = NULL`. La confirmación se protege con una URL firmada (`signed`) en lugar de exigir login; la vista es genérica para no mostrar líneas ni datos personales. Los detalles del flujo están en 04-carrito-y-pedidos.md.*

Estructura relevante del proyecto

```
app/
├── Carrito.php             Carrito de compra sobre la sesión (clave `carrito`)
├── Enums/                  Rol, EstadoPedido, MetodoPago, TipoMovimiento
├── Http/
│   ├── Controllers/        Auth (Breeze), Profile, ProductoController, CarritoController,
│   ├── Middleware/         EnsureUserHasRole
│   └── Requests/           LoginRequest, ProfileUpdateRequest, StoreProductoRequest, Updat
├── Models/                 User, Producto, Pedido, DetallePedido, Insumo, MovimientoInvent
└── View/Components/        AppLayout, GuestLayout
bootstrap/app.php           Registro del alias de middleware `role`
database/
├── factories/              Una factory por modelo
├── migrations/             Esquema de datos
└── seeders/                DatabaseSeeder → AdminUserSeeder, ProductoSeeder
resources/
├── css/app.css             Tailwind v4, fuente Instrument Sans
├── js/
│   ├── app.js              Alpine + arranque del visor
│   └── visor-3d.js         Three.js, GLTFLoader, OrbitControls, limpieza GPU
└── views/                  Breeze (auth, profile), productos (index, show), carrito (index
routes/
├── web.php                 Catálogo (/ y /productos/{producto}), carrito, pedidos, panel y
└── auth.php                Rutas de autenticación de Breeze
storage/app/public/         Modelos 3D servidos vía public/storage (vacío en el repo)
tests/                      Feature (Auth, Admin, Profile, ProductoCatalogo, Carrito, Pedid
```

Convenciones de esta documentación

- El idioma es español técnico; los nombres de clases, tablas y rutas se mantienen tal como existen en el código.
- Los diagramas Mermaid se renderizan directamente en visores compatibles (GitHub, GitLab, VS Code).
- Las referencias a requerimientos siguen la nomenclatura del SRS del proyecto: RF para requerimientos funcionales y RNF para no funcionales.